

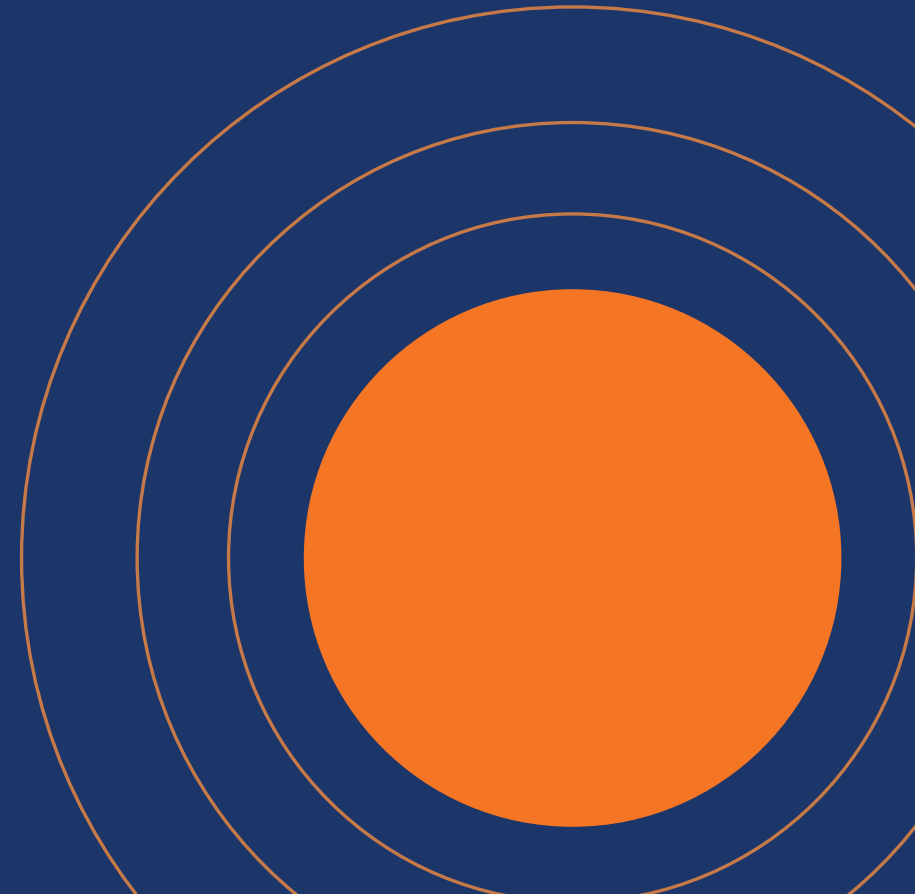


Optimization Theory & Applications

Week 1 — Introduction to Optimization & Problem Formulation

Dr. Taha Yiğit Alkan · Department of Computer Engineering

[Fall / 2026-2027] · CSE 413





What We'll Cover Today

■ PART I · FOUNDATIONS

- What is an optimization problem?
- The three ingredients: variables, objective, constraints and the feasible region
- Local vs. global optima
- A taxonomy of problems and why the class dictates the algorithm

■ PART II · MODELING & LAB

- A repeatable recipe for formulating any problem
- Problem I : minimizing the energy of a circuit
- Problem II: maximizing the throughput of a scheduler
- Lab : visualizing surfaces & feasible regions

Python · NumPy · Matplotlib

Optimization Is the Engine of Modern Computing

Behind almost every engineered system is a quantity someone wants to make as large — or as small — as possible.

■ Machine Learning & AI

Train a model = minimize a loss over millions of parameters.

$\min \text{ loss}(\text{weights})$

■ Networks & Systems

Route traffic to minimize latency and congestion across links.

$\min \text{ delay}(\text{flows})$

■ VLSI & Hardware

Place cells / size gates to minimize chip area, wirelength, power.

$\min \text{ area} \cdot \text{power}$

■ Scheduling & Resources

Assign tasks to processors to maximize throughput.

$\max \text{ throughput}$

The Anatomy of an Optimization Problem

Standard form

$$\text{minimize} \quad f_0(\mathbf{x})$$

$$\text{subject to} \quad f_i(\mathbf{x}) \leq 0, \quad i = 1, \dots, m$$

$$h_j(\mathbf{x}) = 0, \quad j = 1, \dots, p$$

with $\mathbf{x} \in \mathbb{R}^n$ the optimization variable.

■ Decision variable $\mathbf{x}$

the unknowns you get to choose

■ Objective $f_0(\mathbf{x})$

the single number you minimize

■ Constraints f_i, h_j

the limits a solution must respect

■ Feasible set

all $\mathbf{x}$ satisfying every constraint

■ Optimal point $\mathbf{x}^*$ / value p^*

the best feasible $\mathbf{x}$, and its objective value

Maximizing f is the same as minimizing $-f$ — so “minimize” loses no generality.

Decision Variables: What You Control

- The unknowns you are free to choose. The answer the solver returns.
- Collected into a vector $x = (x_1, \dots, x_n)$.
- Dimension n can be tiny (2) or enormous (10^9 in deep learning).

■ Key idea

If you cannot change it, it is a parameter (data). Not a decision variable.

Domain	Variables	Type
Neural net	weights & biases	continuous
Network routing	flow on each link	continuous
Path selection	use link? (0 / 1)	binary
VLSI placement	cell coordinates	continuous
Scheduling	task → slot	integer



The Objective: What “Best” Means

- A function $f(x)$ that maps every decision to a single real number. Its cost or quality.
- We minimize a cost (energy, error, delay) or maximize a utility (throughput, accuracy).
- It must be quantifiable: “make it good” is not an objective; “minimize mean-squared error” is.
- Many real systems have several goals at once. Multi-objective.

$$\min Z_{SP} = \underbrace{w_1 \cdot \text{Cost}_{\text{op}} + w_2 \cdot \text{Cost}_{\text{env}}}_{\text{Operational Costs}} + \underbrace{w_3 \cdot \text{Penalty}_{\text{svc}}}_{\text{Service Penalties}}$$

■ Multi-objective?

Combine goals with weights, or keep one as the objective and turn the rest into constraints. Full treatment later in the term.

■ Sign convention

$$\max f(x) \equiv \min -f(x)$$

So we standardize on minimization.



Constraints & the Feasible Region

- **Constraints are the rules a solution must obey. They carve the feasible region out of $\mathbb{R}^n$.**
- Inequality $f_i(x) \leq 0$ — a region / half-space (e.g., power $\leq$ budget).
- Equality $h_j(x) = 0$ — a surface (e.g., flow in = flow out).
- Feasible region = the set of all points satisfying every constraint at once.
- A constraint is active (binding) if it holds with equality at the optimum. These are the ones that “matter.”

■ Feasible

at least one point satisfies all constraints

■ Infeasible

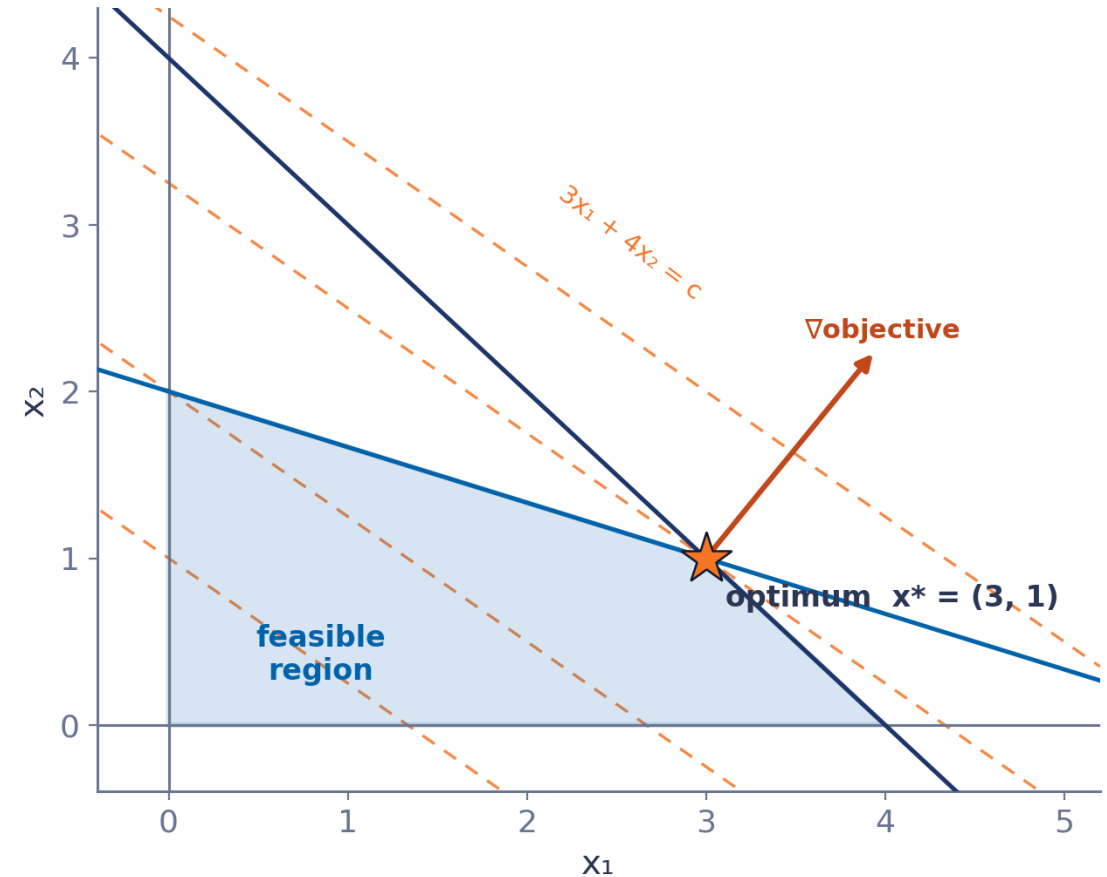
no point can satisfy them. the model is over-constrained

■ Unbounded

the objective improves forever. a constraint is missing

A Geometric Picture in 2-D

- Two decision variables x_1, x_2 , so we can draw the whole problem.
- Shaded polygon = feasible region (the overlap of all constraints).
- Dashed lines = objective contours (level sets: equal-value points).
- Slide the contour in the improving direction (the gradient) until it is about to leave the feasible set.
- The optimum lands on a vertex — a preview of why Simplex works (Week 3).



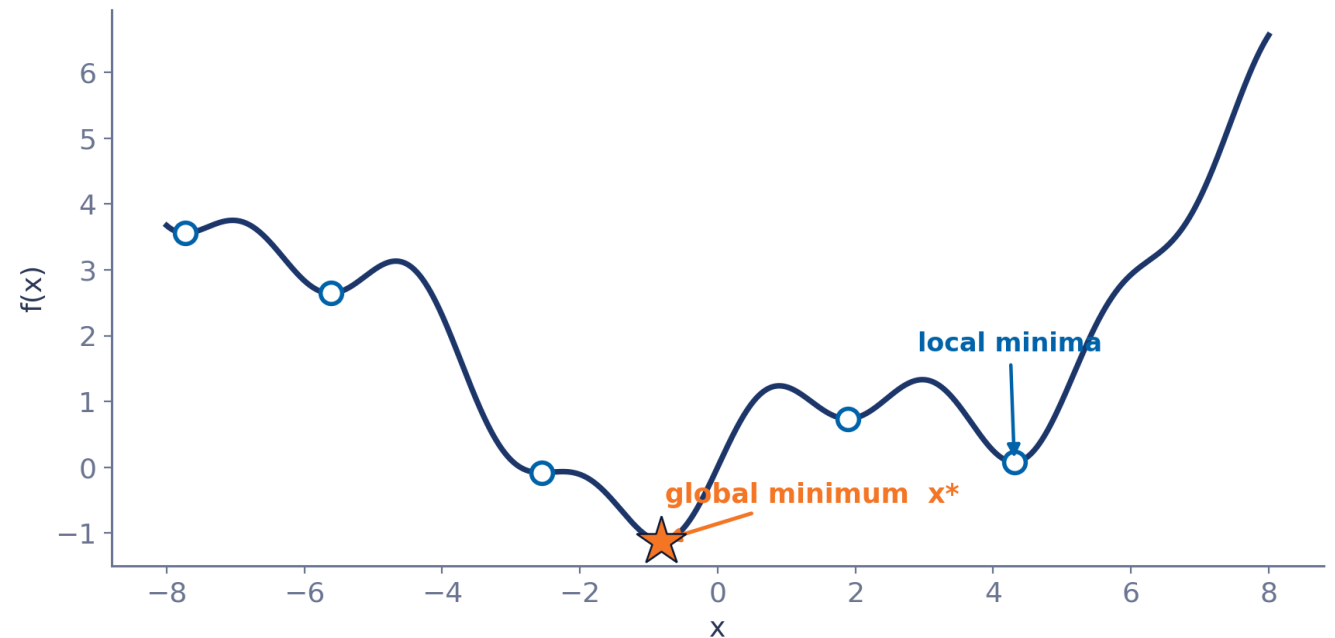
Local vs. Global Optima

Global minimum

$f(x^*) \leq f(x)$ for ALL feasible x .

Local minimum

$f(x^*) \leq f(x)$ only for feasible x NEAR x^* .



Most algorithms only guarantee a local optimum. Convexity is what makes local = global.

A Map of Optimization Problems

Every problem sits at a point along several axes. Where it sits decides how hard it is — and which algorithm applies.

1 Continuous ↔ Discrete

Real-valued knobs, or whole-number choices?

2 Linear ↔ Nonlinear

Are the objective and constraints straight lines, or curved?

3 Convex ↔ Non-convex

One valley, or many? (the great watershed)

4 Unconstrained ↔ Constrained

Free to roam $\mathbb{R}^n$, or fenced into a feasible region?



Continuous vs. Discrete

■ Continuous

- Variables live in $\mathbb{R}$ (real numbers).
- Calculus applies. Gradients & Hessians exist.
- Examples: voltages, weights, flows, positions.
- Workhorse: gradient-based methods.
- Generally the “easier” world.

■ Discrete / Integer

- Variables in $\mathbb{Z}$ or $\{0, 1\}$.
- No derivatives. the space is combinatorial.
- Examples: assignments, routing, scheduling.
- Workhorse: branch-and-bound, search, DP.
- Often NP-hard. Exact solutions can be costly.

Mixed-integer problems have both. Relaxing integers to reals gives bounds — a key trick (Week 12).

Linear vs. Nonlinear

■ Linear

- Objective & constraints are linear: $c^T x$, $Ax \leq b$.
- Linear Programs (LPs).
- Solved at massive scale. Millions of variables.
- Optimum sits at a vertex of the feasible polytope.
- Methods: Simplex, interior-point.

■ Nonlinear

- Objective or constraints are curved.
- Uses gradients and Hessians.
- Examples: least squares, neural nets, power $\propto V^2$.
- Richer modeling. But generally harder.
- Methods: gradient, Newton, SQP.

Linearity $\neq$ easy and nonlinearity $\neq$ hard. The deeper question is the next axis: convexity.

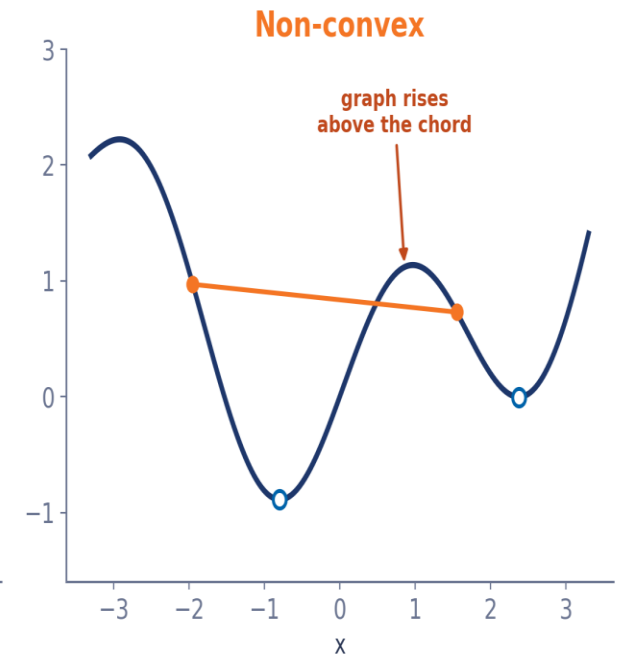
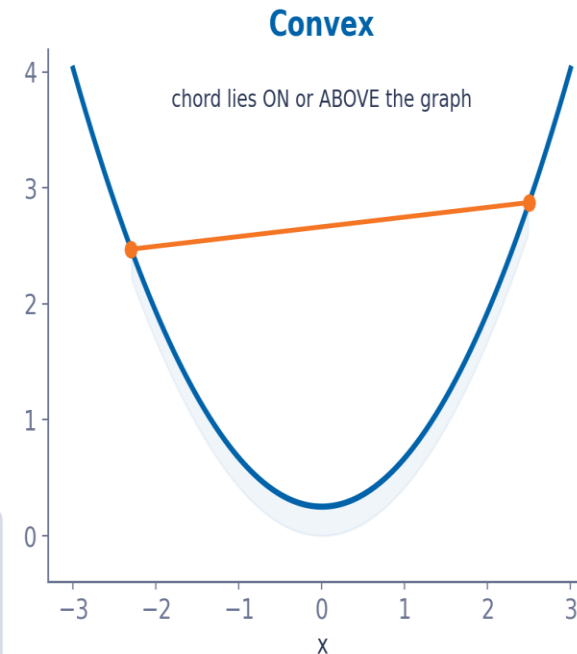
Convex vs. Non-convex

- **Convex set:** the segment between any two points stays inside.
- **Convex function:** every chord lies on or above the graph.
- Convex $\Rightarrow$ every local minimum is global, and we can certify optimality (duality).
- Non-convex (deep nets, combinatorial): many local minima; globally NP-hard.



“The great watershed in optimization is not between linearity and nonlinearity, but convexity and non-convexity.”

— R. T. Rockafellar



Unconstrained vs. Constrained

■ Unconstrained

- Minimize $f(x)$ over all of $\mathbb{R}^n$.
- Optimum is a stationary point: $\nabla f(x^*) = 0$.
- Methods: gradient descent, Newton.
- The natural setting for model fitting / training.

■ Constrained

- Must satisfy constraints. Stay in the feasible set.
- Optimum often lies ON the boundary (recall the polygon).
- Tools: Lagrange multipliers & KKT conditions.
- Real engineering: power, timing, area budgets always bind.

A common strategy: convert constrained $\rightarrow$ unconstrained via penalty or barrier terms.

Why the Problem Class Dictates the Algorithm

Step 1 is always to classify the problem. The class tells you which tool to reach for and whether an exact answer is even possible.

Problem class	Typical method(s)	Course week
Linear program	Simplex · interior-point	3–4
Unconstrained smooth	Gradient descent · Newton / quasi-Newton	5–6
Constrained nonlinear	Lagrange / KKT · SQP	7
Convex	Interior-point · CVXPY modeling	8–9
Large-scale / ML	SGD · momentum · Adam · proximal	10–11
Integer / combinatorial	Branch-and-bound · dynamic programming	12–13
Intractable / very large	Heuristics · metaheuristics	14



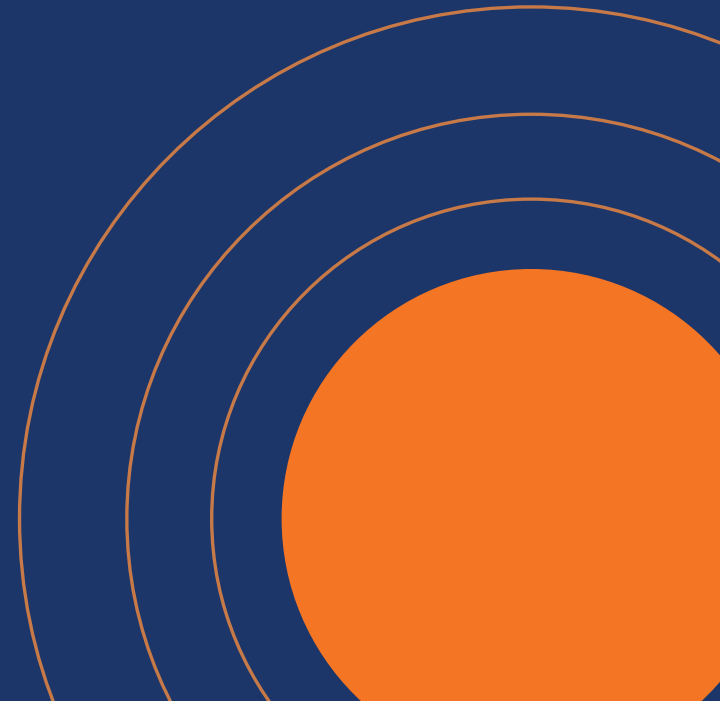
AKDENİZ UNIVERSITY
DEPARTMENT OF COMPUTER ENGINEERING



PART II

From Words to Math

Translating engineering problems into precise, solvable optimization models.



A Recipe for Formulating Any Problem

1

Identify the decision variables

What, exactly, are you free to choose?

2

Define the objective

What does “best” mean — minimize or maximize what?

3

List the constraints

Physical limits, budgets, capacities, logical rules.

4

Pin down types & data

Continuous or integer? Which numbers are fixed parameters?

5

Classify the problem

Place it on the axes — LP, convex, integer, ...

6

Choose a solver / algorithm

The class points to the right tool.



Minimizing the Energy of a Circuit

■ The problem, in words

- A digital block has gates with supply voltages $v_1, \dots, v_n$.
- Dynamic energy per gate grows like $C_i v_i^2$ (the CV^2 law).
- Lower voltage saves energy but makes gates slower.
- The block must still meet a timing deadline T .
- Each voltage has physical lower/upper limits.

■ The model

$$\begin{aligned} &\text{minimize} && E(v) = \sum C_i v_i^2 \\ &\text{subject to} && \text{delay}(v) \leq T \\ &&& v_{\min} \leq v_i \leq v_{\max}, \quad i = 1, \dots, n \end{aligned}$$

■ **Problem class:** continuous · nonlinear · convex (if delay is modeled convexly) · constrained → convex / NLP methods; relates to geometric programming.



Maximizing the Throughput of a Scheduler

■ The problem, in words

- N tasks must run on M processors.
- Goal: complete as many tasks as possible.
- Each task is assigned to at most one processor.
- No processor may exceed its capacity C_j .
- Decision per pair: run task i on processor j , yes or no.

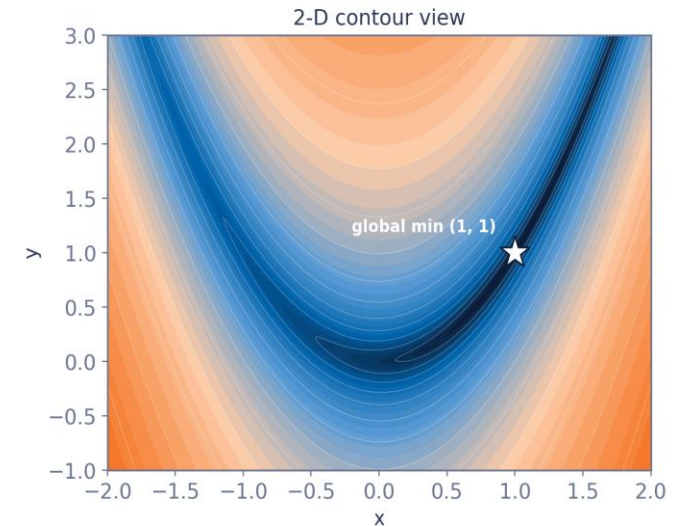
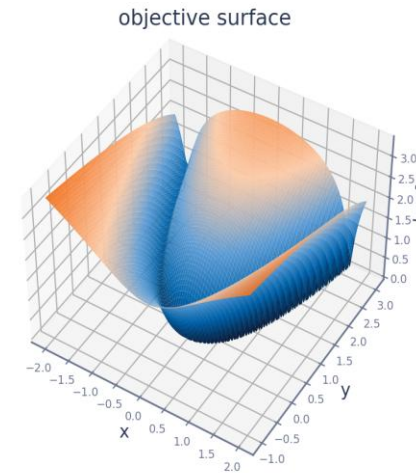
■ The model

$$\begin{aligned}
 &\text{maximize} && \sum_{i,j} x_{ij} \\
 &\text{subject to} && \sum_j x_{ij} \leq 1, \quad \forall i \\
 & && \sum_i w_i x_{ij} \leq C_j, \quad \forall j \\
 & && x_{ij} \in \{0, 1\}
 \end{aligned}$$

- **Problem class:** discrete (binary) · linear · constrained → an integer program (ILP); generally NP-hard, solved by branch-and-bound or heuristics.

Lab: Seeing Optimization in Python

- Set up Python / Jupyter; vectors & grids with NumPy.
- Define objective functions as plain Python callables.
- Visualize objective surfaces (3-D) and contour plots.
- Draw a 2-D feasible region from linear constraints.
- **Peek ahead: call `scipy.optimize.minimize`.**



```
import numpy as np, matplotlib.pyplot as plt
f = lambda x, y: (1-x)**2 + 100*(y-x**2)**2      # Rosenbrock
X, Y = np.meshgrid(np.linspace(-2,2,200), np.linspace(-1,3,200))
plt.contourf(X, Y, np.log10(f(X,Y)+1), levels=40)
plt.plot(1, 1, 'w*'); plt.show()                # global minimum
```

WRAP-UP

Summary & Looking Ahead

KEY TAKEAWAYS

- ✓ Every problem = variables + objective + constraints.
- ✓ Local $\neq$ global, and convexity is the bridge.
- ✓ Classify first: the class chooses the algorithm.
- ✓ Modeling is the core engineering skill.

BEFORE NEXT WEEK

- Read Boyd & Vandenberghe Ch. 1 · Kochenderfer & Wheeler Ch. 1
- Finish the Python / Matplotlib lab notebook
- HW 0: formulate two CE problems in standard form

■ NEXT WEEK

Mathematical Foundations. Gradients, Hessians, and convex sets & functions: the machinery behind every algorithm.

